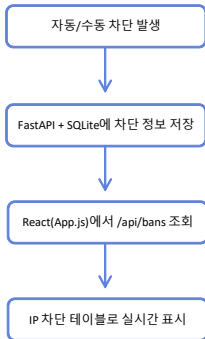


1. IP 차단 대시보드 (React + FastAPI + SQLite)

<작동 방식>



IP 차단 관리 새로고침

내 IP: 203.247.169.218 상태: 정상

IP 주소 (예: 1.2.3.4) 3600 이유 (선택) 차단 추가

IP	이유	생성	만료	TTL(초)	작성자	작업
현재 활성 차단이 없습니다.						

▶ 로그인 보안·IPS·수동 차단으로 발생한 모든 차단 IP를 한 화면에서 조회·연장·해제할 수 있는 관리 대시보드

이 대시보드는 로그인 보안·IPS 자동 차단·수동 차단 등을 시각화하여 모든 차단 이벤트를 한 화면에서 관리하기 위해 제작했습니다

1. IP 차단 대시보드 (주요 코드)

```
// 차단 목록 조회
const fetchBans = async () => {
  const res = await fetch(
    `${API_BASE}/api/bans?active_only=${onlyActive ? 'true' : 'false'}`
  );
  const data = await res.json();
  if (data.success) setBans(data.items || []);
};

// 수동 차단 추가
const handleAddBan = async () => {
  await fetch(`${API_BASE}/api/bans`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      ip: newBanIp,
      ttl_seconds: Number(newBanTtl) || 3600,
      reason: newBanReason || 'manual_ban',
      actor: 'manual'
    })
  });
  fetchBans();
};

// TTL 연장 / 차단 해제
const handleExtend = (ip) => fetch(`${API_BASE}/api/bans/${ip}/extend`, { method: 'POST' });
const handleUnban = (ip) => fetch(`${API_BASE}/api/bans/${ip}/unban`, { method: 'POST' });
```

<대시보드 역할>

자동 차단된 IP + 수동 차단 IP를한 화면에서 관리 IP, 이유, 생성·만료 시간(TTL), 차단 주체(actor)를 테이블로 표시

<동작 방식>

GET /api/bans

→ SQLite(bans 테이블)에서 차단 목록 조회 후 화면에 표시

POST /api/bans

→ 수동 차단 추가(입력한 IP를 ipset + DB에 등록)

POST /api/bans/{ip}/extend

→ TTL 연장(+30분 등)

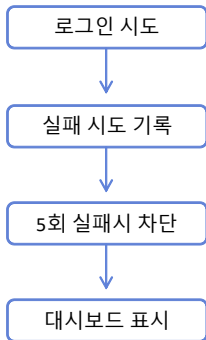
POST /api/bans/{ip}/unban

→ 차단 해제(ipset + DB에서 삭제)

2.로그인 보안 기능 - Python 기반 실패 누적 차단 시스템



<차단 방식>



자동 차단 → 관리 화면 반영

AWS 모니터링 대시보드 로그인

접속 차단됨: 203.247.169.218

5회 실패 시 IP가 차단됩니다.

IP 차단 관리

새로고침

내 IP: *

상태: 정상

IP 주소 (예: 1.2.3.4)

3600

이유 (선택)

차단 추가

IP	이유	생성	만료	TTL(초)	적정자	작업
58.78.106.132	login_fail_x5	2025. 12. 5. 오후 11:51:05	2025. 12. 6. 오전 12:51:05	3600	auto-login-guard	삭제

▶ 자동 차단된 IP가 실시간으로 표시되는 대시보드

2.로그인 보안 기능 - Python 기반 실패 누적 차단 시스템 - 주요 코드

로그인 실패 누적 및 차단 트리거 코드 설명 1

```
# 로그인 실패 누적
cnt = record_login_failure(client_ip)

# 5회 이상 실패 시 자동 차단
if cnt >= LOGIN_FAIL_LIMIT:
    ttl = DEFAULT_BAN_TTL
    validate_ip(client_ip)

    ipset_add(client_ip, ttl)    # ◆ ipset에 즉시 추가 (Fail2Ban 대비 빠름)

    create_ban_record(
        client_ip,
        ttl,
        reason=f"login_fail_x{cnt}",
        actor="auto-login-guard",
    )

    clear_login_failures(client_ip)

    raise HTTPException(
        status_code=403,
        detail=f"로그인 실패 {cnt}회로 인해 IP가 {ttl}s 동안 차단되었습니다.",
    )
```

1. record_login_failure로 실패 횟수 누적 관리
→ DB에 실패 기록을 저장하며 일정 시간 안에 반복 실패만 누적
2. LOGIN_FAIL_LIMIT 달성 시 자동 차단 실행
→ 5회 이상이면 바로 차단.
3. ipset_add()로 kernel-level 차단 실행
→ Fail2Ban보다 빠르고 서버 부하 적음.
4. create_ban_record()로 DB에 TTL 포함 저장
→ 대시보드에서 TTL 관리/조회 가능.
5. clear_login_failures로 누적 초기화
→ 차단 후에는 실패 기록 리셋.

로그인 실패 누적 및 차단 트리거 코드 설명 2

```
# 차단 레코드 생성 (DB에 TTL 포함 저장)
def create_ban_record(ip: str, ttl_seconds: int, reason: str, actor: str = "system"):
    created = now_utc()
    expires = created + timedelta(seconds=ttl_seconds)

    conn = get_db()
    cur = conn.cursor()
    cur.execute("""
        INSERT INTO bans(ip, reason, created_at, expires_at, ttl_seconds, actor)
        VALUES(?, ?, ?, ?, ?, ?)
    """, (ip, reason, iso(created), iso(expires), ttl_seconds, actor))
    ...
    return row
```

create_ban_record – 차단 정보 생성

-> 차단된 IP에 대해
생성 시간(created_at),
만료 시간(expires_at),
TTL(ttl_seconds),
차단 이유(reason),
자동/수동(actor)
를 함께 저장.

```
# TTL 만료된 IP 정리
def purge_expired() -> int:
    conn = get_db()
    cur = conn.cursor()

    cur.execute("SELECT DISTINCT ip FROM bans WHERE expires_at <= ?", (now_utc().isoformat(),))
    ips = [r["ip"] for r in cur.fetchall()]

    cur.execute("DELETE FROM bans WHERE expires_at <= ?", (now_utc().isoformat(),))
    conn.commit()
    conn.close()

    # ipset에서도 제거
    for ip in ips:
        ipset_del(ip)

    return len(ips)
```

purge_expired – TTL 만료 자동 정리

-> 만료된 IP를 DB에서 자동 삭제
동시에 ipset에서도 제거
관리자가 신경 쓰지 않아도 되는 자동 해제 시스템

3. IPS 자동 차단

IPS란? : 서버에 대한 해킹 시도가 발생할 때 자동으로 감지하고 차단해주는 보안 시스템

<동작 방법>

- 1) Suricata가 네트워크 트래픽을 검사
- 2) 공격 패턴(Port Scan, SQL Injection 등) 탐지
- 3) 각 공격에 해당하는 severity(위험도) 판정
- 4) 우리 Python IPS 엔진에서 정책 적용
 - severity 0~2 → 자동 차단(ipset + DB 저장)
 - severity 3 → 경고만
- 5) 차단된 IP는 실시간 대시보드에 표시
- 6) 관리자가 TTL 연장 / 해제 / 수동 차단 가능

실시간 IDS 보안 알림 (3건)

2025-12-08 05:57:21

Custom SQL Injection Attempt

SRC: 3.36.23.86 | DEST: 172.31.42.112

높음

2025-12-08 05:57:41

Custom XSS Attempt

SRC: 3.36.23.86 | DEST: 172.31.42.112

높음

2025-12-08 05:57:58

SSH SYN

SRC: 203.247.169.218 | DEST: 172.31.42.112

낮음

<탐지하는 5가지 주요 공격>

- 1) Port Scan — 문이 열려 있는지 확인하는 정찰
해커가 서버의 여러 포트를 두드려보며 “어디로 들어갈 수 있을까?” 탐색하는 행위.
→ 낮은 위험도 → 경고만
- 2) SYN Scan — 매우 빠른 속도로 문을 두드리는 정찰
포트스캔의 고속 버전. 공격 전에 어디가 열려있는지 확인하는 단계.
→ 낮은 위험도 → 경고만
- 3) SSH Brute Force — 비밀번호 계속 찍어 맞추기
SSH(22번 포트)로 로그인 패스워드를 반복적으로 시도하는 공격.
→ 로그인 보안 시스템에서 우선 차단 처리
- 4) SQL Injection — 웹사이트를 속여 DB에 침투하려는 공격
URL에 “or 1=1” 같은 코드를 넣어 DB를 조작하려는 해킹.
→ 고위험 → 자동 6시간 차단
- 5) XSS 공격 — 웹페이지에 악성 스크립트 심기
<script>alert(1)</script> 같은 코드를 넣어 사용자의 쿠키·세션을 탈취하는 공격.
→ 고위험 → 자동 6시간 차단

3. IPS 자동 차단

<IDS 룰 설정>

```
alert tcp any any -> $HOME_NET any \
(msg:" SYN Scan"; \
flow:stateless; flags:S; \
detection_filter:track by_src, count 40, seconds 3; \
classtype:attempted-recon; priority:3; sid:2000001; rev:3;)

alert tcp any any -> $HOME_NET any \
(msg:" Port Scan Detected"; \
flow:stateless; flags:S; \
detection_filter:track by_src, count 20, seconds 10; \
classtype:attempted-recon; priority:3; sid:2000002; rev:3;)

alert tcp any any -> $HOME_NET 22 \
(msg:" SSH Brute Force Detected"; \
flow:stateless; flags:S; \
detection_filter:track by_src, count 25, seconds 60; \
classtype:attempted-admin; priority:3; sid:2000003; rev:3;)

alert http any any -> $HOME_NET any \
(msg:" Custom SQL Injection Attempt"; \
flow:to_server,established; \
content:"select"; nocase; http_uri; \
pcrc:"/(bUNION\b|s+SELECT\b|BOR\b|s+1\s*=\s*1\b|'\s*or\s*1\s*=\s*1|'\s*or\s*1\s*=\s*1|'\s*or\s*1\s*=\s*1)/Ui"; \
classtype:web-application-attack; priority:1; sid:2000004; rev:3;)

alert http any any -> $HOME_NET any \
(msg:" Custom XSS Attempt"; \
flow:to_server,established; \
content:"<script"; http_uri; nocase; \
pcrc:"/(<script\b|javascript:|onerror\s*=\s*|onload\s*=\s*|alert\s*\()/Ui"; \
classtype:web-application-attack; priority:1; sid:2000005; rev:3;)
```

1) Port Scan 탐지

60초 동안 50회 이상 TCP 접속 시도 발생 시 탐지
정찰(Reconnaissance) 단계 공격으로 분류
낮은 위험도 → 경고 알림만 발생

2) SYN Scan 탐지

10초 동안 SYN 패킷 200회 초과 시 탐지
매우 빠른 포트 스캐닝(공격 전 탐색 단계)
낮은 위험도 → 경고 알림만 발생

3) SSH Brute Force 탐지

(해당 부분은 로그인 보안에서 먼저 탐지후 대응 → 1시간 차단)

4) SQL Injection 탐지

URI에select,union,or 1=1등 SQLi 패턴 포함 시 탐지
데이터베이스 정보 유출 위험
높은 위험도 → 6시간 차단

5) XSS 공격 탐지

URI에<script>,javascript:,alert()등 포함 시 탐지
웹 페이지 변조, 쿠키 탈취 등 심각한 피해 가능
높은 위험도 → 6시간 차단

3. IPS 자동 차단

실시간 IDS 보안 알림 (5건)

2025-12-08 07:58:22
SSH SYN
SRC: 210.119.103.197 | DEST: 172.31.42.112

2025-12-08 07:58:19
SSH SYN
SRC: 210.119.103.197 | DEST: 172.31.42.112

2025-12-08 07:58:17
SSH SYN

낮음

낮음

낮음

IP 차단 관리

새로고침

내 IP: 210.119.103.197

상태: 정상

IP 주소 (예: 1.2.3.4)

3600

이유 (선택)

차단 추가

IP	이유	생성	만료	TTL(초)	작성 자	작업
----	----	----	----	--------	---------	----

현재 활성 차단이 없습니다.

저위험 공격은 위험도 낮음으로 표시되고 차단은 적용되지 않음

실시간 IDS 보안 알림 (2건)

2025-12-08 07:54:47
Custom SQL Injection Attempt
SRC: 3.36.23.86 | DEST: 172.31.42.112

2025-12-08 07:54:27
Custom XSS Attempt
SRC: 3.36.23.86 | DEST: 172.31.42.112

높음

높음

IP 차단 관리

새로고침

내 IP: 210.119.103.197

상태: 정상

IP 주소 (예: 1.2.3.4)

3600

이유 (선택)

차단 추가

IP	이유	생성	만료	TTL(초)	작성 자	작업
3.36.23.86	suricata:Custom SQL Injection Attempt	2025. 12. 8. 오전 7:54:47	2025. 12. 8. 오후 1:54:47	21600	ids	+30초 삭제

고위험 공격 (SQL injection,XSS공격)은 위험도 높음으로 표시 된 후 차단이 실행되었음